

The Ultimate Guide to the `sed` Command

Introduction

`sed` (Stream Editor) is a powerful command-line utility for parsing and transforming text. It reads text from a file or standard input, performs operations on it line by line, and outputs the modified text to standard output. It's a fundamental tool for shell scripting, system administration, and data manipulation.

This guide provides a comprehensive overview of `sed`, from basic usage to advanced recipes.

Basic Syntax

The basic syntax of `sed` is:

```
sed [OPTIONS] 'SCRIPT' [INPUT_FILE...]
```

- **[OPTIONS]**: Command-line options, like `-i` for in-place editing or `-f` to specify a script file.
 - **'SCRIPT'**: The `sed` commands to execute, typically in the format `'address command flags'`.
 - **[INPUT_FILE...]**: The file(s) to process. If no file is provided, `sed` reads from standard input.
-

Core Concepts

1. Pattern Space and Hold Space

- **Pattern Space**: This is the active buffer where `sed` holds the current line being processed. Most commands operate on the pattern space. After all commands in the script have been executed on the line, the pattern space is printed to standard output, and the next line is read into it.
- **Hold Space**: This is an auxiliary buffer. You can use it to copy or move text from the pattern space for later use. It's key for complex, multi-line operations.

2. The `sed` Script

A `sed` script consists of one or more commands. Each command has up to three parts:

1. **Address**: Specifies which line(s) to operate on. This is optional. If omitted, the command is applied to *every* line.
2. **Command**: A single letter representing the action to take (e.g., `s` for substitute, `d` for delete).
3. **Flags/Options**: Extra parameters for the command (e.g., `g` for global substitution).

Part 1: Basic Operations

Substitution: The s Command

The most common `sed` command is `s` for substitution.

```
sed 's/old_text/new_text/' filename.txt
```

Example 1: Basic Substitution Replace the first occurrence of `Bash` with `Perl` on each line.

```
# Create a sample file
echo "Bash Scripting is fun. Bash is powerful." > languages.txt

# Run the command
sed 's/Bash/Perl/' languages.txt
# Output: Perl Scripting is fun. Bash is powerful.
```

Substitution Flags

- **g (Global):** Replace *all* occurrences of the pattern on a line, not just the first.

```
sed 's/Bash/Perl/g' languages.txt
# Output: Perl Scripting is fun. Perl is powerful.
```

- **i (Case-Insensitive):** Match the pattern regardless of case (GNU `sed` extension).

```
echo "bash scripting" | sed 's/Bash/Perl/i'
# Output: Perl scripting
```

- **Numbered Flag:** Replace only a specific occurrence.

```
# Replace the second occurrence of 'Bash'
sed 's/Bash/Perl/2' languages.txt
# Output: Bash Scripting is fun. Perl is powerful.
```

Deletion: The d Command

The `d` command deletes the entire line from the pattern space.

Example 2: Delete Lines Containing a Pattern Delete any line containing the word `Android`.

```
# Create a sample file
printf "iOS\nAndroid\nLinux\n" > os.txt

# Run the command
sed '/Android/d' os.txt
```

```
# Output:
# iOS
# Linux
```

In-Place Editing: The -i Option

To modify a file directly instead of printing to standard output, use the `-i` option.

Warning: This modifies the file directly. It's good practice to create a backup.

```
# Create a backup file with a .bak extension
sed -i.bak 's/Linux/GNU/Linux/' os.txt

# Modify without a backup (use with caution)
sed -i 's/iOS/Apple iOS/' os.txt
```

Note: BSD sed (on macOS) requires an extension for -i, e.g., sed -i '' ... for no backup.

Part 2: Addressing (Selecting Lines)

You can restrict which lines a command affects by providing an address.

1. By Line Number

- **A single line number:** `sed '3d' filename.txt` (deletes the 3rd line).
- **A range of lines:** `sed '2,4s/old/new/' filename.txt` (substitutes on lines 2 through 4).
- **Start line to end:** `sed '5,$d' filename.txt` (deletes from line 5 to the end of the file).
- **Every Nth line:** `sed '1~2d' filename.txt` (deletes every second line, starting from line 1).

2. By Regular Expression (Regex)

This is the most powerful way to select lines. The address is a regex enclosed in slashes (`/pattern/`).

- **Lines containing a pattern:** `sed '/ERROR/d' log.txt` (deletes all lines with “ERROR”).
- **A range of lines defined by patterns:** `sed '/start_pattern/,/end_pattern/d'` (deletes everything from the first line matching `start_pattern` to the first line matching `end_pattern`).
- **Lines *not* containing a pattern:** Use the `!` operator. `sed '/Linux/!d'` (deletes all lines that *do not* contain “Linux”).

Example 3: Conditional Substitution Only substitute Count with 100 on lines that also contain CSE.

```
sed '/CSE/s/Count/100/' student_counts.txt
```

Part 3: Regular Expressions in sed

sed uses Basic Regular Expressions (BRE) by default. Key metacharacters include:

Character	Description	Example
.	Matches any single character.	s/l.ve/love/
*	Matches zero or more preceding characters.	s/a*/b/
^	Anchors the pattern to the start of a line.	s/^Start/Begin/
\$	Anchors the pattern to the end of a line.	s/end.\$/end./
[]	A character set, matches any char inside.	s/[abc]/X/
[^]	A negated character set.	s/[^0-9]/g (del non-digits)
\(\)	Capture Group. Captures a sub-pattern.	s/(foo\)bar/\1/ -> foo
&	In replacement, refers to the entire match.	s/word/[\&]/ -> [word]
\1, \2..	In replacement, refers to a capture group.	s/(w1\) \ (w2\)/\2 \1/ (swap words)

POSIX Character Classes

For more reliable matching across different systems, use POSIX character classes.

Class	Description	Equivalent
[[[:alnum:]]	Alphanumeric characters	[A-Za-z0-9]
[[[:alpha:]]	Alphabetic characters	[A-Za-z]
[[[:digit:]]	Digits	[0-9]
[[[:lower:]]	Lower-case letters	[a-z]
[[[:upper:]]	Upper-case letters	[A-Z]
[[[:space:]]	Whitespace (space, tab, newline, etc.)	[\t\n\r\f\v]
[[[:blank:]]	Space and tab	[\t]
[[[:punct:]]	Punctuation characters	!"#\$%&'()*+,-./:;<=>?@[^_`{ }~

Example 4: Remove Trailing Whitespace

```
sed 's/[[[:blank:]]]*$//' filename.txt
```

Part 4: Advanced Recipes & Scripts

Recipe 1: Remove the Last Character of Each Line

A classic use case. The `.` matches any character, and `$` anchors it to the end of the line.

```
sed 's/.$//' filename.txt
```

Recipe 2: Insert/Append Text

- **i (insert)**: Add text on a new line *before* the specified line.
- **a (append)**: Add text on a new line *after* the specified line.

```
# Insert a header at the beginning of the file  
sed '1i # My File Header' filename.txt
```

```
# Append a footer to the end of the file  
sed '$a # End of File' filename.txt
```

Recipe 3: Swapping Two Words

Use capture groups to remember parts of the pattern.

```
echo "first second" | sed 's/\([^ ]*\) \([^ ]*\)/\2 \1/'  
# Output: second first
```

Recipe 4: Add Commas to Numbers

This uses a loop within the `sed` script. `:a` creates a label `a`. `ta` branches to label `a` if the last `s` command was successful.

```
echo "1234567" | sed -e ':a' -e 's/\([0-9]\{3\}\)/\1,\2/;ta'  
# Output: 1,234,567
```

Recipe 5: Multi-line Operations

By default, `sed` operates on one line at a time. The `N`, `P`, `D` commands work with multiple lines in the pattern space. - `N`: Appends the next line of input into the pattern space, separated by a newline `\n`. - `P`: Prints the pattern space up to the first newline. - `D`: Deletes the pattern space up to the first newline and restarts the script.

Example: Replace a multi-line block Replace the block “Linux” followed by “Android” with “Ubuntu” followed by “Android Lollipop”.

```
sed '$!N;s/Linux\nAndroid/Ubuntu\nAndroid Lollipop/;P;D' os.txt
```

Recipe 6: Using a Script File (-f)

For complex scripts, it's cleaner to put the commands in a separate file.

rules.sed:

```
# This is a comment
s/PHP/Hypertext Preprocessor/g
s/independent/dependent/
```

Command:

```
sed -f rules.sed input.txt
```

Recipe 7: Print Specific Lines (-n and p)

By default, `sed` prints the pattern space after executing the script. The `-n` option suppresses this. You can then use the `p` command to explicitly print only certain lines.

```
# Print only lines 5 to 10
sed -n '5,10p' filename.txt

# Print only lines containing "SUCCESS"
sed -n '/SUCCESS/p' log.txt
```

Recipe 8: Case Conversion (\U, \L)

With GNU `sed`, you can change the case of matched patterns. - `\U`: Converts the replacement to uppercase. - `\L`: Converts the replacement to lowercase. - `\u`: Converts the next character to uppercase. - `\l`: Converts the next character to lowercase.

Example: Capitalize the first letter of every word

```
echo "hello world" | sed 's/\b(.\)/\u\1/g'
# Output: Hello World
```

Example: Convert a whole line to uppercase

```
echo "all lower" | sed 's/.*/\U&/'
# Output: ALL LOWER
```

Conclusion

`sed` is an incredibly versatile and efficient tool for text manipulation. While tools like `awk` and `perl` offer more advanced programming features, `sed` excels at simple, stream-based editing tasks. Mastering `sed` is a significant step toward proficiency in the Linux/Unix command-line environment.

50 Sed Command Examples

This section provides 50 practical examples of **sed** commands, adapted from the “50 sed Command Examples” guide by LinuxHint.

1. Basic Text Substitution

Replace the first occurrence of a pattern on a line.

```
echo "Bash Scripting Language" | sed 's/Bash/Perl/'  
# Output: Perl Scripting Language
```

2. Replace All Instances in a Line

Use the **g** flag to replace all occurrences of a pattern on a line.

```
echo "Python is easy to use. Python is easy to learn." | sed 's/Python/Perl/g'  
# Output: Perl is easy to use. Perl is easy to learn.
```

3. Replace the Nth Occurrence

Replace only a specific occurrence of a pattern on a line.

```
echo "Python is easy to use. Python is easy to learn." | sed 's/Python/Perl/2'  
# Output: Python is easy to use. Perl is easy to learn.
```

4. Replace Only the Last Occurrence

Replace the last occurrence of a pattern on a line.

```
echo "Bash Programming Language. Python Programming Language. Perl Programming Language." |  
# Output: Bash Programming Language. Python Programming Language. Perl Scripting Language.
```

5. Replace the First Match in a File

Replace only the first occurrence of a pattern in the entire file.

```
printf "Python is a very popular language.\nPython is easy to use." | sed '1s/Python/Perl/'  
# Output:  
# Perl is a very popular language.  
# Python is easy to use.
```

6. Replace the Last Match in a File

Replace only the last occurrence of a pattern in the entire file.

```
printf "Python is a very popular language.\nPython is easy to use." | sed '$s/Python/Bash/'  
# Output:  
# Python is a very popular language.  
# Bash is easy to use.
```

7. Escaping Slashes in File Paths

To replace file paths, use a different delimiter or escape the slashes.

```
echo "/home/ubuntu/code/perl/add.pl" | sed 's\\/\\/\\/\\/g'
# Output: \home\ubuntu\code\perl\add.pl
```

8. Extract Filename from a Full Path

Remove the directory path and keep only the filename.

```
echo "/home/ubuntu/temp/myfile.txt" | sed 's/.*\\/'
# Output: myfile.txt
```

9. Conditional Substitution

Substitute text only if another pattern is found on the same line.

```
printf "CSE Count\\nEEE Count" | sed '/CSE/s/Count/100/'
# Output:
# CSE 100
# EEE Count
```

10. Substitute If a Pattern Is NOT Found

Use ! to negate the address match.

```
printf "CSE Count\\nEEE Count" | sed '/CSE/!s/Count/80/'
# Output:
# CSE Count
# EEE 80
```

11. Add Text Before and After a Match

Use capture groups to add text around a matched pattern.

```
echo "Bash language" | sed 's/(Bash)/Learn \\1 programming/'
# Output: Learn Bash programming language
```

12. Delete Matching Lines

Delete all lines that contain a specific pattern.

```
printf "Windows\\nLinux\\nAndroid\\nOS" | sed '/OS/d'
# Output:
# Windows
# Linux
# Android
```


13. Delete a Line and the Next N Lines

Delete the line containing a pattern and the 2 lines that follow.

```
printf "Windows\nLinux\nAndroid\nOS" | sed '/Linux/,+2d'
# Output:
# Windows
```

14. Delete Trailing Spaces

Remove all spaces and tabs from the end of each line.

```
echo "Hello World   " | sed 's/[[:blank:]]*$//'
# Output: Hello World
```

15. Delete Lines Containing a Pattern Twice

Delete lines where a pattern appears at least twice.

```
printf "PHP is a server-side scripting language.\nPHP is open-source and PHP is case-sensitive\nPHP is a server-side scripting language."
# Output:
# PHP is a server-side scripting language.
```

16. Delete Empty Lines

Remove all empty lines from a file.

```
printf "Line 1\n\nLine 3" | sed '/^$/d'
# Output:
# Line 1
# Line 3
```

17. Delete Non-Printable Characters

Remove non-printable characters like tabs.

```
echo -e "Hello\tWorld" | sed 's/^[[:print:]]//g'
# Output: HelloWorld
```

18. Append Text to a Matching Line

If a line contains a pattern, append text to the end of it.

```
printf "Windows\nLinux" | sed '/Windows/s/$/ 10/'
# Output:
# Windows 10
# Linux
```

19. Insert a Line Before a Match

Insert a new line of text before a line that contains a pattern.

```
printf "Line 1\nLine 3" | sed '/Line 3/i \nLine 2'
# Output:
# Line 1
#
# Line 2
# Line 3
```

20. Insert a Line After a Match

Insert a new line of text after a line that contains a pattern.

```
printf "Line 1\nLine 3" | sed '/Line 1/a \nLine 2'
# Output:
# Line 1
#
# Line 2
# Line 3
```

21. Append Text to Non-Matching Lines

Append text to all lines that do *not* contain a specific pattern.

```
printf "Windows\nLinux\nOS" | sed '/Linux/!s$/ Operating System/'
# Output:
# Windows Operating System
# Linux
# OS Operating System
```

22. Delete Non-Matching Lines

Delete all lines that do *not* contain a specific pattern.

```
printf "HTML\nCSS\nJavaScript" | sed '/CSS/!d'
# Output: CSS
```

23. Duplicate a Matched Word

Duplicate a word or pattern on the same line.

```
echo "Python is a very popular language." | sed 's/very /&&/'
# Output: Python is a very very popular language.
```

24. Substitute from a List of Strings

Use `awk` and `sed` to perform substitutions based on a mapping file.

```

# list1.txt
# 1001:Jafar Ali
# 1023:Nir Hossain

# list2.txt
# 1001 CSE GPA-3.63
# 1023 CSE GPA-3.11

awk -F: '{printf("s/%s/%s\n", $1, $2)}' list1.txt > commands.sed
sed -f commands.sed list2.txt
# Output:
# Jafar Ali CSE GPA-3.63
# Nir Hossain CSE GPA-3.11

```

25. Replace a String with a Newline

Insert a newline character into the output.

```

echo "Bash Perl Python" | sed 's/Python/Added Text\n/'
# Output:
# Bash Perl Added Text
#

```

26. Remove Newlines and Add Commas

Join all lines in a file into a single line separated by commas.

```

printf "Windows\nLinux\nAndroid" | sed -z 's/\n/,/g'
# Output: Windows, Linux, Android,

```

27. Split a Comma-Separated String into Multiple Lines

Replace commas with newlines.

```

echo "Kaniz,Fatema,30th" | sed 's/,/\n/g'
# Output:
# Kaniz
# Fatema
# 30th

```

28. Case-Insensitive Deletion

Delete lines containing a pattern, ignoring case.

```

printf "Windows\nlinux\nAndroid" | sed '/linux/Id'
# Output:
# Windows
# Android

```

29. Case-Insensitive Replacement

Replace a pattern, ignoring case.

```
echo "I like bash programming" | sed 's/Bash/PHP/i'
# Output: I like PHP programming
```

30. Convert Match to Uppercase

Convert the matched pattern to uppercase.

```
echo "I like linux" | sed 's/\(linux\)/\U\1/'
# Output: I like LINUX
```

31. Convert Match to Lowercase

Convert the matched pattern to lowercase.

```
echo "I like LINUX" | sed 's/\(LINUX\)/\L\1/'
# Output: I like linux
```

32. Convert Entire Line to Lowercase

Convert all characters on a line to lowercase.

```
echo "ALL CAPS" | sed 's/.*/\L&/'
# Output: all caps
```

33. Prepend a Currency Symbol to Numbers

Add a character before the first number on a line.

```
printf "HDD 100\nMonitor 80" | sed 's/\([0-9]\)/\$/1/'
# Output:
# HDD $100
# Monitor $80
```

34. Add Commas to Numbers (Thousands Separator)

Format numbers with commas.

```
echo "5098673" | sed -e ':a' -e 's/\(.*[0-9]\)\([0-9]\{3\}\)/\1,\2/;ta'
# Output: 5,098,673
```

35. Replace Tabs with Spaces

Replace each tab character with a specific number of spaces.

```
echo -e "1\t2\t3" | sed 's/\t/ /g'
# Output: 1 2 3
```

36. Replace Multiple Spaces with a Tab

Replace four consecutive spaces with a single tab character.

```
echo "1    2    3" | sed 's/    /\t/g'
# Output: 1\t2\t3
```

37. Truncate Lines to a Specific Length

Cut off lines that are longer than 80 characters.

```
echo "This is a very long line that will be truncated." | sed 's/\(^.\{1,20\}\).*$/\1/'
# Output: This is a very long
```

38. Append Text After a Regex Match

Append text after a specific word or pattern.

```
echo "hello, how are you?" | sed 's/\(hello\)/\1 John/'
# Output: hello John, how are you?
```

39. Append Text After the Nth Match

Append text after the second occurrence of a pattern.

```
echo "PHP is great. PHP is fast." | sed 's/\(PHP\)/\1 (New Text added)/2'
# Output: PHP is great. PHP (New Text added) is fast.
```

40. Run a Multi-Line Script from a File

Execute a series of `sed` commands stored in a file.

```
# sedcmd:
# s/PHP/ASP/g
# s/independent/dependent/

printf "PHP is independent." | sed -f sedcmd
# Output: ASP is dependent.
```

41. Match and Replace a Multi-Line Pattern

Search for a pattern that spans multiple lines.

```
printf "Linux\nAndroid\nOS" | sed '$!N;s/Linux\nAndroid/Ubuntu\nAndroid Lollipop;/P;D'
# Output:
# Ubuntu
# Android Lollipop
# OS
```

42. Swap the Order of Two Words

Reverse the order of two words on a line.

```
echo "perl python" | sed 's/\([^ ]*\) *\([^ ]*\)/\2 \1/'  
# Output: python perl
```

43. Execute Multiple sed Commands

Use the `-e` option to run multiple `sed` commands in a single line.

```
echo "Ubuntu Centos Debian" | sed -e 's/Ubuntu/Kubuntu/;' -e 's/Centos/Fedora/'  
# Output: Kubuntu Fedora Debian
```

44. Combine sed with Other Commands

Pipe the output of other commands to `sed` for processing.

```
cat os.txt | sed 's/Linux/Fedora/' | sed 's/Windows/Windows 10/i'
```

45. Insert an Empty Line After Each Line

Double-space a file.

```
printf "Line 1\nLine 2" | sed 'G'  
# Output:  
# Line 1  
#  
# Line 2
```

46. Replace Alphanumeric Characters with a Space

Remove all letters and numbers, leaving only symbols.

```
echo "ID: [101]" | sed 's/[A-Za-z0-9]//g'  
# Output: : []
```

47. Use & to Refer to the Matched String

Use the `&` character to include the entire matched pattern in the replacement string.

```
echo "Linux is powerful" | sed 's/^L.*/Matched String is - &/'  
# Output: Matched String is - Linux is powerful
```

48. Switch Pairs of Words in a File

Swap the first and second words on each line.

```
printf "PHP ASP\nMySQL Oracle" | sed 's/\([^ ]*\) *\([^ ]*\)/\2 \1/'  
# Output:
```

```
# ASP PHP
# Oracle MySQL
```

49. Capitalize the First Character of Each Word

Change the first letter of each word to uppercase.

```
echo "i like bash programming" | sed 's/\b\([a-z]\)/\u\1/g'
# Output: I Like Bash Programming
```

50. Print Line Numbers

Prepend the line number to each line of output.

```
printf "Windows\nLinux\nAndroid" | sed =
# Output:
# 1
# Windows
# 2
# Linux
# 3
# Android
```